Day3 Task

```
import java.util.Scanner;
class Person {
    private String name;
    private int age;
// Default constructor
    public Person() {
        this.name = "Unknown";
        this.age = 0;
    }
 // Parameterized constructor
    public Person(String name, int age) {
        this.name = name;
        setAge(age); // reuse setter so validation applies
    }

    // Getters
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    // Setters with validation (the benefit of encapsulation)
```

```
    public void setName(String name) {
        if (name != null && !name.trim().isEmpty()) {
            this.name = name;
        } else {
            System.out.println("Invalid name. Value not changed.");
        }
    }

    public void setAge(int age) {
        if (age > 0 && age < 120) {
            this.age = age;
        } else {
            System.out.println("Invalid age. Value not changed.");
        }
    }

    public void displayDetails() {
        System.out.println("Name : " + name);
        System.out.println("Age  : " + age);
    }
}

// ------------------------------------------------------------
// Student: INHERITS from Person
// ------------------------------------------------------------
class Student extends Person {
    private int rollNo;
    private String course;
```

```
private double marks;

public Student() {
    super(); // calls Person()
    this.rollNo = 0;
    this.course = "Not assigned";
    this.marks = 0.0;
}

public Student(String name, int age, int rollNo, String course, double marks) {
    super(name, age); // calls Person(String, int)
    this.rollNo = rollNo;
    this.course = course;
    setMarks(marks);
}

public int getRollNo() {
    return rollNo;
}

public void setRollNo(int rollNo) {
    this.rollNo = rollNo;
}

public String getCourse() {
    return course;
}
```

```
public void setCourse(String course) {
    this.course = course;
}

public double getMarks() {
    return marks;
}

public void setMarks(double marks) {
    if (marks >= 0 && marks <= 100) {
        this.marks = marks;
    } else {
        System.out.println("Invalid marks. Must be between 0 and 100.");
    }
}

public String calculateGrade() {
    if (marks >= 75) return "Distinction";
    else if (marks >= 60) return "First Class";
    else if (marks >= 50) return "Second Class";
    else if (marks >= 40) return "Pass";
    else return "Fail";
}

// METHOD OVERRIDING: replaces Person's displayDetails()
@Override
public void displayDetails() {
    System.out.println("----- Student Details -----");
```

```
        super.displayDetails(); // reuse parent's code
        System.out.println("Roll No : " + rollNo);
        System.out.println("Course  : " + course);
        System.out.println("Marks   : " + marks);
        System.out.println("Grade   : " + calculateGrade());
    }
}

// ---------------------------------------------------------------
// Employee: INHERITS from Person
// ---------------------------------------------------------------
class Employee extends Person {
    private int empId;
    private String designation;
    private double salary;

    public Employee() {
        super();
        this.empId = 0;
        this.designation = "Not assigned";
        this.salary = 0.0;
    }

    public Employee(String name, int age, int empId, String designation, double salary) {
        super(name, age);
        this.empId = empId;
        this.designation = designation;
        setSalary(salary);
```

```
}

public int getEmpId() {
    return empId;
}

public void setEmpId(int empId) {
    this.empId = empId;
}

public String getDesignation() {
    return designation;
}

public void setDesignation(String designation) {
    this.designation = designation;
}

public double getSalary() {
    return salary;
}

public void setSalary(double salary) {
    if (salary >= 0) {
        this.salary = salary;
    } else {
        System.out.println("Invalid salary. Value not changed.");
    }
```

```
    }

    public double calculateAnnualSalary() {
        return salary * 12;
    }

    public void giveRaise(double percent) {
        if (percent > 0) {
            salary += salary * percent / 100;
            System.out.println("Raise applied. New monthly salary: " + salary);
        }
    }

    @Override
    public void displayDetails() {
        System.out.println("----- Employee Details -----");
        super.displayDetails();
        System.out.println("Emp ID      : " + empId);
        System.out.println("Designation : " + designation);
        System.out.println("Monthly Sal : " + salary);
        System.out.println("Annual Sal  : " + calculateAnnualSalary());
    }
}

// ------------------------------------------------------------
// Main class: takes input from the user and creates the objects
// ------------------------------------------------------------
public class Main {
```

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // ----- Student input -----
    System.out.println("=== Enter Student Details ===");
    System.out.print("Name: ");
    String sName = sc.nextLine();
    System.out.print("Age: ");
    int sAge = sc.nextInt();
    System.out.print("Roll No: ");
    int rollNo = sc.nextInt();
    sc.nextLine(); // clear leftover newline
    System.out.print("Course: ");
    String course = sc.nextLine();
    System.out.print("Marks (0-100): ");
    double marks = sc.nextDouble();

    // Parameterized constructor
    Student student = new Student(sName, sAge, rollNo, course, marks);

    // ----- Employee input -----
    sc.nextLine(); // clear leftover newline
    System.out.println("\n=== Enter Employee Details ===");
    System.out.print("Name: ");
    String eName = sc.nextLine();
    System.out.print("Age: ");
    int eAge = sc.nextInt();
    System.out.print("Employee ID: ");
```

```
int empId = sc.nextInt();
sc.nextLine();
System.out.print("Designation: ");
String designation = sc.nextLine();
System.out.print("Monthly Salary: ");
double salary = sc.nextDouble();

Employee employee = new Employee(eName, eAge, empId, designation, salary);

// ----- Display -----
System.out.println();
student.displayDetails();
System.out.println();
employee.displayDetails();

// ----- Encapsulation in action: update via setters -----
System.out.println("\n=== Update Employee ===");
System.out.print("Enter raise percentage: ");
double percent = sc.nextDouble();
employee.giveRaise(percent);

System.out.println("\nTrying to set invalid age using setter:");
employee.setAge(-5); // rejected by validation

// ----- Default constructor demo -----
System.out.println("\n=== Object created with default constructor ===");
Student blank = new Student();
blank.displayDetails();
```

```
        // ----- Polymorphism: parent reference, child object -----
        System.out.println("\n=== Polymorphism demo ===");
        Person[] people = { student, employee };
        for (Person p : people) {
            p.displayDetails(); // calls the overridden version of each class
            System.out.println();
        }

        sc.close();
    }
}
```